

A) Training Setup

1. Environment

We use the stand task environment from Homework 2, following the same task design structure with the components required in IsaacLab: observations, actions, rewards, resets, and terminations. In Project 1, this environment is used to train an RL policy that can keep the humanoid robot stand stably for more than 20 seconds under disturbances.

The environment includes:

- observations for both policy and critic,
- joint-position actions for 13 controlled joints,
- reward terms encouraging upright and stable standing,
- reset randomization for initial state variation,
- and termination conditions for falls and episode timeout.

The environment runs correctly and produces valid observations, actions, rewards, and terminations at every step.

1.1 Policy observations (PolicyCfg)

The policy observation vector is the concatenation of:

1. **Base angular velocity (3D)** in the base frame
2. **Projected gravity (3D)** in the base frame
3. **Velocity commands (3D)**, generated by `zero_velocity_command`, which are all zeros because this is a standing task
4. **Joint position relative offsets (13D)** for the controlled joints
5. **Joint velocity relative values (13D)** for the controlled joints
6. **Previous action (13D)**

Therefore, the total policy observation dimension is:

$$3 + 3 + 3 + 13 + 13 + 13 = 48D$$

1.2 Critic observations (CriticCfg)

The critic uses all policy observations plus additional privileged information:

- **Base linear velocity (3D)**
- **Base height (1D)**

- **Feet contact state (2D)** using force thresholded contact information
- **Feet speed in x-y (4D)** for the left and right feet
- **Feet height (2D)**

So, the critic observation dimension is:

$$\text{Policy 48D} + \text{extras } (3 + 1 + 2 + 4 + 2) = 60\text{D}$$

The actor-critic design allow the critic to access richer state information during training while keep the policy observation compact.

2. Action space

The robot is controlled through **13 action joints**, including both legs and the waist yaw joint:

- 6 joints for the left leg
- 6 joints for the right leg
- 1 waist yaw joint

Actions are implemented using `mdp.JointPositionActionCfg`, where the action represents a normalized joint position command around the default standing pose.

The configuration uses:

- `joint_names = ACTION_JOINTS_13`
- `scale = ACTION_SCALE_BY_JOINT`
- `use_default_offset = True`

The scale for each joint is computed from the safe symmetric range around the default pose using the predefined joint limits (`ACTIONS_MIN_13`, `ACTIONS_MAX_13`).

This ensures that commanded joint targets remain within a valid range around the nominal standing posture.

3. Reward terms and weights

The reward design encourages upright, stable, and quiet standing while penalizing unstable motion and failure states.

Term	Description	Weight
alive	alive bonus	+2.0
upright	uprightness from projected gravity	+10.0
height	base height tracking to 0.70 m	+6.0
pose	similarity to default 13 joint pose	+3.0
terminating	termination penalty	-50.0
lin_vel	penalize base linear velocity L2	-0.2
ang_vel	penalize base angular velocity L2	-0.2
joint_vel	penalize joint velocity L2	-0.01
action_rate	penalize action rate L2	-0.01

feet_still	penalize horizontal foot motion	-0.8
undesired_contact	penalize undesired body contact	-2.0
waist_yaw_hold	penalize waist yaw deviation	-0.3

Interpretation

- The numbers were chosen by priority and scale. +10.0 for upright makes staying upright the strongest positive objective, since this is the core goal of the task. +6.0 for height is slightly smaller but still strong, because the robot should keep a correct standing height without letting height dominate orientation. +3.0 for pose is lower, since the robot should stay near the default pose but still keep enough freedom to balance. +2.0 for alive gives a constant survival incentive, but it is not large enough to outweigh poor posture.
- -50.0 for terminating is much larger than the other penalties so that falling is always much worse than making small motion errors. -0.2 for lin_vel and ang_vel is moderate: large enough to reduce drifting and spinning, but not so large that the robot becomes too stiff to recover. -0.01 for joint_vel and action_rate is small because these are regularization terms; they should smooth motion, not dominate learning. -0.8 for feet_still is stronger because foot sliding is a clear standing failure behavior. -2.0 for undesired_contact is also stronger because unwanted body contact is close to failure. -0.3 for waist_yaw_hold is moderate, enough to discourage upper-body twisting without completely removing waist motion.

4. Resets

The environment uses two reset events.

4.1 reset_base

This reset randomizes the robot root state with moderate variation:

- x position in $[-0.02, 0.02]$
- y position in $[-0.02, 0.02]$
- roll in $[-0.01, 0.01]$
- pitch in $[-0.01, 0.01]$
- yaw fixed at 0
- initial linear velocity fixed at zero

4.2 reset_joints

This reset randomizes the 13 controlled joints using:

- position_range = (0.97, 1.03)
- zero joint velocity

Purpose

These reset events introduce moderate variation in initial conditions and help prevent overfitting to a single deterministic starting pose. This supports the project requirement that the robot remain stable under initial condition changes.

5. Terminations

The environment terminates an episode under the following conditions:

- **Time-out** using `mdp.time_out`
- **Fall by height** using `mdp.fall_by_height` with `min_height = 0.5`
- **Bad orientation** using `mdp.bad_orientation` with `limit_angle = 1.0`
- **Undesired contact** using `mdp.undesired_contact_termination`

These conditions are consistent with the standing task goal: the episode ends when the robot falls, tilts excessively, or makes invalid contact.

6. Robustness condition

To check robustness, the environment uses reset randomization and a disturbance during the episode. These make the task harder than standing from one fixed starting pose and help check if the learned policy can recover balance after perturbations.

The reset randomization described in the **Resets** section already improves robustness by varying the initial root pose and joint configuration at the beginning of each episode.

In addition, the environment includes a disturbance event called **push_robot**. A periodic disturbance is applied during the episode using `mdp.push_by_setting_velocity` with:

- interval range: 6.0–10.0 s
- velocity perturbation:
 - x in $[-0.3, 0.3]$
 - y in $[-0.3, 0.3]$

This disturbance acts as a push-like perturbation by changing the robot root horizontal velocity, forcing the policy to recover balance after being disturbed.

7 RL Algorithm

We train the humanoid standing policy using an actor-critic reinforcement learning algorithm, specifically Proximal Policy Optimization (PPO).

Runner configuration

The training runner is defined in `HumanoidN1PPORunnerCfg` with the following settings:

- **Seed:** 42
- **Device:** `cuda:0`

- **Experiment name:** PJ1
- **Run name:** ppo_test
- **Rollout length:** num_steps_per_env = 64
- **Maximum iterations:** 1200
- **Checkpoint save interval:** every 10 iterations
- **Observation groups:** {"policy": ["policy"], "critic": ["critic"]}
- **Action clipping:** 1.0
- **Number of environments:** 1024
- **Simulation time step:** 0.001 s
- **Decimation:** 10
- **Control step:** 0.01 s
- **Episode length:** 20.0 s = 2000 control steps
- **Total batch size per update:** 1024 x 64 = 65536
- **Mini-batch size:** 65536 / 16 = 4096

Actor - critic network

The policy and value function are modeled using separate multilayer perceptrons under an actor-critic architecture.

- **Initial action noise std:** 1.0
- **Noise parameterization:** log
- **Actor observation normalization:** enabled
- **Critic observation normalization:** enabled
- **Actor hidden dimensions:** [512, 256, 128]
- **Critic hidden dimensions:** [512, 256, 128]
- **Activation function:** ELU

The actor network maps policy observations to continuous joint commands, while the critic network estimates the value function from critic observations.

PPO hyperparameters

The PPO algorithm uses the following hyperparameters:

Hyperparameter	Value
Learning rate	3e-4
Schedule	adaptive
Desired KL	0.01
Discount factor γ	0.99
GAE lambda λ	0.95
Entropy coefficient	0.01
Max gradient norm	1.0
Value loss coefficient	1.0
Clipped value loss	True

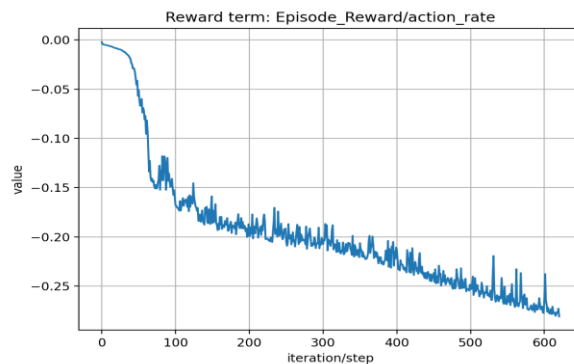
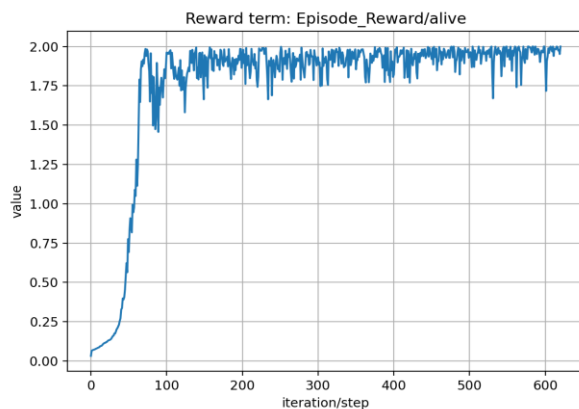
Clip parameter	0.2
Learning epochs per update	8
Mini batches per update	16

Interpretation

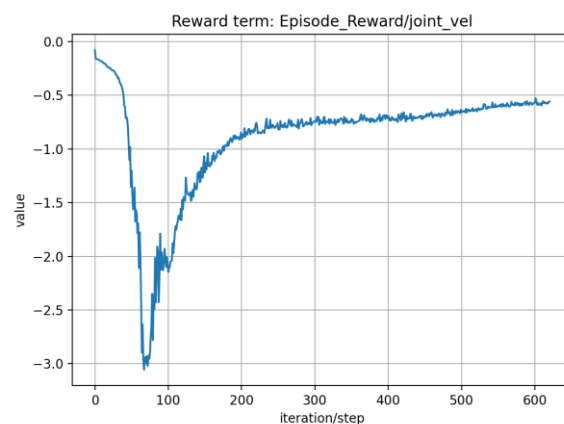
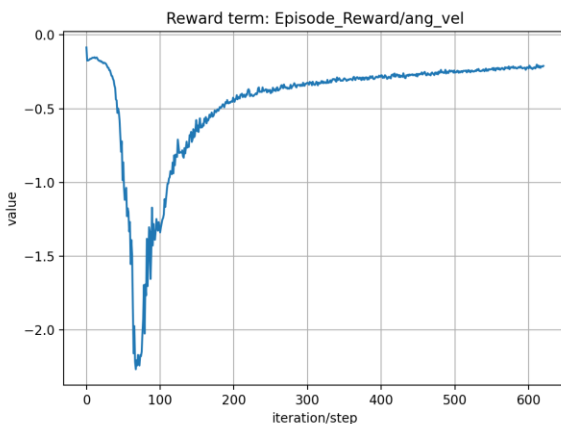
The PPO configuration is designed to balance learning stability and exploration:

- The learning rate $3e-4$ is a standard moderate value: large enough to make progress, but small enough to avoid unstable updates. The adaptive schedule and desired_kl = 0.01 control update size so the policy does not change too aggressively in one iteration.
- The entropy coefficient 0.01 keeps some exploration during training.
- The PPO clip 0.2 is the standard choice to prevent overly large policy changes
- Finally, 8 learning epochs and 16 mini-batches per update were chosen to reuse collected data efficiently

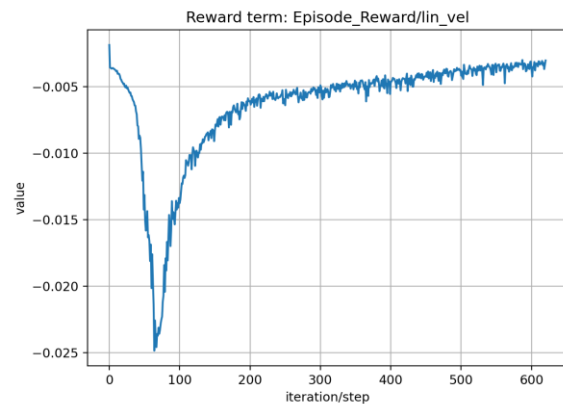
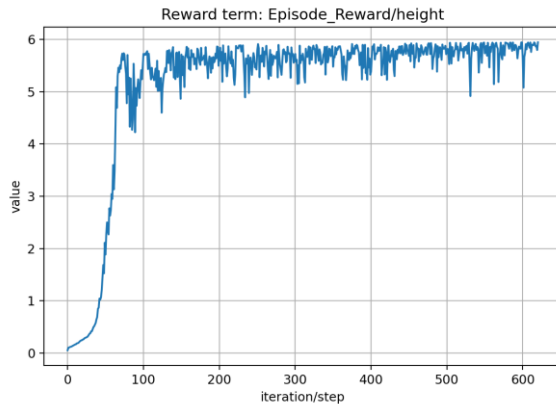
B) Results



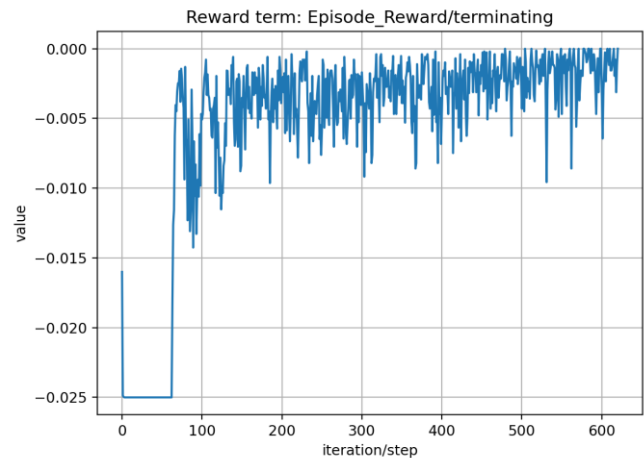
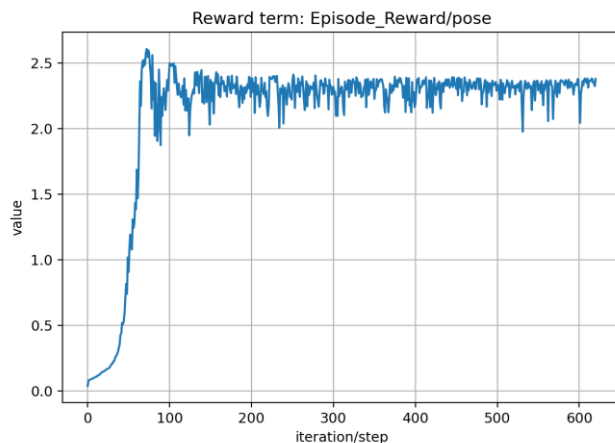
Alive reward increases rapidly and stabilizes close to maximum value, show that the policy learned to avoid early termination and maintain standing for a long duration. In contrast, the action rate penalty becomes more negative over training, which suggests that the learned controller uses more active corrective joint to maintain balance under reset random and push disturbances.



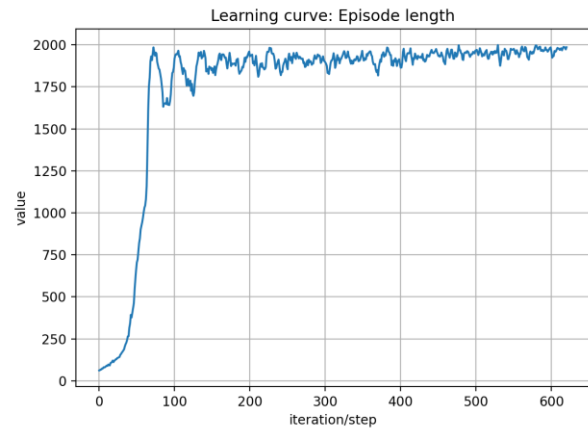
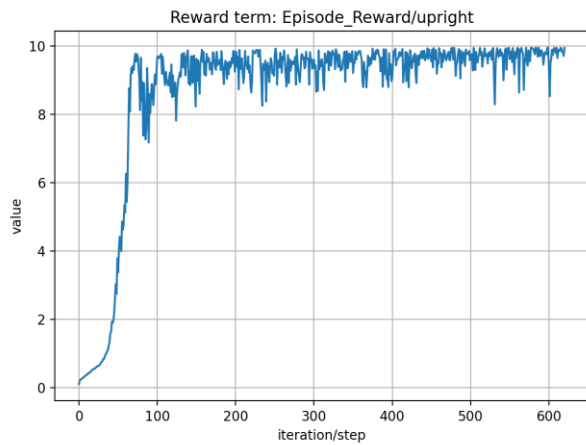
Both `ang_vel` and `joint_vel` rewards drop sharply early in training, showing that the robot at first makes large angular and joint motions while trying to find a stable position. After that, both curves steadily improve and become less negative, which means the policy learns to reduce unnecessary body rotation and joint movement, which suggest the robot now is becoming more stable.



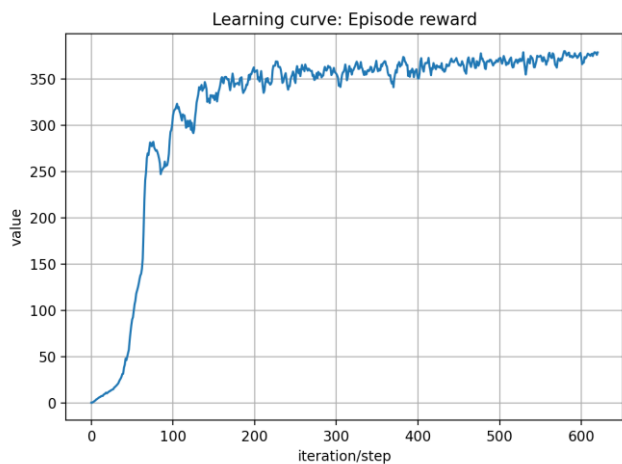
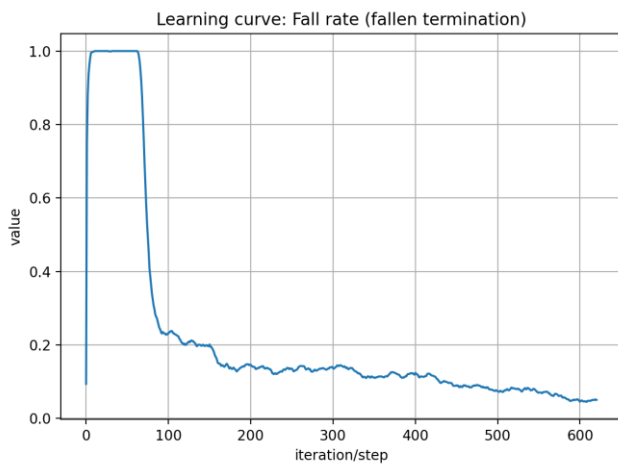
The height reward rises quickly and stays close to its maximum value, which shows that the policy learns to keep the base near the target height. Early in training, the policy explores unstable behaviors, so the robot moves around more and the penalty drops. Later, as the policy learns to stand more stably, base motion is reduced, so the curve rises again.



The pose reward increases quickly and then remains high, which indicates that the policy learns to keep the robot close to the default standing posture. The terminating reward becomes much closer to zero over training, meaning failure events and early terminations become less frequent. Some noise remains because your environment still includes reset randomization and dynamic corrections, so the robot does not experience exactly the same state in every episode.



Episode length rises quickly and stays near the maximum, showing that the robot learns to survive for most of the episode. The upright reward also increases rapidly and remains close to its maximum, meaning the robot keeps its body well aligned with gravity. The remaining fluctuations come from exploration and small balance corrections, so the robot is mostly stable but still makes occasional adjustments.



In the early stages, the robot often fell because it tried to remain perfectly stiff. However, after the push disturbances were introduced, the policy managed how to learn. So at the same time, the episode reward increases steadily and then remains high, meaning the overall standing performance improves as the policy learns better balance behavior. The remaining fluctuations come from exploration and occasional unstable episodes, but the overall trend clearly indicates successful learning


```
Mean action noise std: 2.09
Mean value_function loss: 0.0184
  Mean surrogate loss: -0.0063
    Mean entropy loss: 22.1348
      Mean reward: 378.71
    Mean episode length: 1985.63
  Episode_Reward/alive: 2.0000
Episode_Reward/upright: 9.9495
  Episode_Reward/height: 5.9439
    Episode_Reward/pose: 2.3783
  Episode_Reward/terminating: 0.0000
    Episode_Reward/lin_vel: -0.0030
      Episode_Reward/ang_vel: -0.2101
        Episode_Reward/joint_vel: -0.5579
          Episode_Reward/action_rate: -0.2809
            Episode_Reward/feet_still: -0.1062
              Episode_Reward/undesired_contact: 0.0000
                Episode_Reward/waist_yaw_hold: -0.0184
                  Episode_Termination/time_out: 0.9499
                    Episode_Termination/fall: 0.0501
                      Episode_Termination/bad_orientation: 0.0000
                        Episode_Termination/undesired_contact: 0.0000
```

The results table shows that the policy learned the standing task quite well. The mean episode length reaches 1985.63, which is very close to the maximum of 2000 steps, and Episode_Termination/time_out = 0.9499 indicates that most episodes end because the time limit is reached rather than because of failure. At the same time, Episode_Termination/fall = 0.0501 means the fall rate is only about 5.01%, corresponding to a success rate of about 94.99%, which exceeds the minimum requirement of 80%.

Regarding the reward breakdown, the main positive rewards are all high and close to their desired values: alive = 2.0000, upright = 9.9495, height = 5.9439, and pose = 2.3783. This indicates that the robot maintains a relatively upright posture, keeps a stable base height, and stays fairly close to the default standing pose. In addition, Episode_Reward/terminating = 0.0000, bad_orientation = 0.0000, and undesired_contact = 0.0000 show that the policy almost never fails excessive tilting or undesired contact.

The negative penalties are still present but remain small, such as lin_vel = -0.0030, ang_vel = -0.2101, joint_vel = -0.5579, action_rate = -0.2809, feet_still = -0.1062, and waist_yaw_hold = -0.0184. This is reasonable, because the robot still needs a certain level of adjustment to maintain balance, especially under random resets and push disturbances. In other words, the policy does not stand completely frozen, but instead learns to make the corrective motions needed to remain stable.

Overall, the trained policy satisfies the project requirements: it can stand for the full 20-second evaluation horizon in most episodes, achieves a success rate of 94.99%, and remains robust under reset randomization and push disturbances